

Week 5: Faster Sampling with DDIM

Theme: 1000 sampling steps is too many. Cut it to 50 with no quality loss.
Time Commitment: 8–10 hours (lighter recovery week after Week 4)
Suggested Split: 3h reading, 5h coding/benchmarking, 1h writeup

Learning Objectives

By the end of this week, you should be able to:

- Articulate the difference between stochastic (DDPM) and deterministic (DDIM) sampling
- Implement DDIM sampling on top of your existing pre-trained DDPM
- Benchmark sampling speed vs. quality trade-offs systematically
- Understand the role of the η (eta) parameter

Primary Resources (Must-Do)

[PAPER] DDIM Paper: “Denoising Diffusion Implicit Models” (Song et al., 2021)

Type: Research Paper | **Time:** 90 min | **Difficulty:** Advanced

URL: <https://arxiv.org/abs/2010.02502>

Why this resource: The original DDIM paper. Surprisingly accessible if you’ve internalized DDPM. The key insight: the same trained model can be sampled deterministically with far fewer steps.

Focus on: Sections 1–4. Skip the proofs in the appendix unless you’re curious. The sampling equation is what you’ll implement.

[BLOG/DOCS] Lilian Weng: DDIM Section

Type: Blog Post | **Time:** 30 min | **Difficulty:** Intermediate-Advanced

URL: <https://lilianweng.github.io/posts/2021-07-11-diffusion-models/#speed-up-diffusion-model-sampling>

Why this resource: Concise companion to the paper. Focuses on the practical implementation differences between DDPM and DDIM sampling.

[CODE] Hugging Face Diffusers: DDIMScheduler Source

Type: Code | **Time:** 30 min | **Difficulty:** Intermediate

URL: https://github.com/huggingface/diffusers/blob/main/src/diffusers/schedulers/scheduling_ddim.py

Why this resource: Production-grade reference implementation. Read the `step()` method carefully — compare it to your DDPM implementation.

Focus on: Lines around the variance calculation and the use of `eta`.

[VIDEO] AI Coffee Break: “DDIM Explained”**Type:** YouTube Video | **Time:** 15 min | **Difficulty:** Intermediate**URL:** <https://www.youtube.com/watch?v=344w5h24-h8>**Why this resource:** Quick, visual summary of why DDIM works. Watch this before diving into the paper for an overview.**This Week’s Deliverable**Add DDIM sampling to your existing DDPM model from Week 4. **No retraining needed** — this is the magic. Code must include:

- A `DDIMScheduler` class with configurable `eta` parameter (where $\eta = 0$ is fully deterministic, $\eta = 1$ recovers DDPM)
- A method to compute the appropriate timestep subset (evenly spaced from $[0, T]$)
- Side-by-side comparison: DDPM at 1000 steps vs. DDIM at 10, 25, 50, and 100 steps
- Wall-clock timing for each setting (use `time.time()` or `torch.cuda.Event`)
- A short markdown report (`ddim_report.md`) with sample grids and a timing table
- Bonus: an interpolation experiment — pick two random noise tensors and linearly interpolate between them, decoding each step. With deterministic sampling, you get smooth transitions.

Key Concepts Checklist

You should be able to confidently answer all of the following:

- Why can DDIM use the same trained model as DDPM?
- What does the η parameter control? What happens at $\eta = 0$ vs. $\eta = 1$?
- Why is interpolation between noise vectors only meaningful with deterministic sampling?
- How does DDIM choose which timesteps to use when running fewer steps?
- What’s the trade-off between number of sampling steps and quality?
- Why doesn’t DDPM produce good samples in 50 steps but DDIM does?

Stretch Resources (Optional)

- **Implement DPM-Solver or DPM-Solver++:** <https://arxiv.org/abs/2206.00927> — even faster than DDIM
- **Try image-to-image generation:** Take an existing image, partially noise it (don’t go to T), then denoise. This is how SDEdit works.
- **Yang Song’s blog on score-based models:** <https://yang-song.net/blog/2021/score/> — unified theoretical view of diffusion
- **Sanity check:** verify that DDIM with $\eta = 1$ and 1000 steps produces samples statistically equivalent to DDPM

Debugging & Reference

- **If DDIM samples are noticeably worse than DDPM at the same step count:** You probably have a sign error in the deterministic update equation
- **If timing isn’t faster:** Verify you’re actually skipping intermediate timesteps, not just iterating with smaller updates
- **Common mistake:** Indexing into the noise schedule arrays with the wrong timestep values when subsampling